

CHESSCOIN

Software Requirements & Architecture Specification

Online Chess Platform • Competitive Leagues • Tournaments • Coaching •
Payments • Anti-Cheat

Target: Responsive Web Application + iOS + Android

Document purpose: Implementation-ready functional, technical, security and deployment specification for the engineering team.

Version 1.0 • September 2026

1. Executive Summary

ChessCoin is a competitive online chess platform inspired by products such as Chess.com. The name “ChessCoin” is a product name only; the system is **not** a cryptocurrency, blockchain, token or wallet product. The platform combines real-time chess, Elo ratings, competitive paid leagues, prize distribution, traditional tournaments, coaching, social features, notifications and a secure administration console.

The architecture should be modular and configuration-driven so that league scoring, eligibility, prize allocation, tournament rules and operational controls can evolve without rewriting core application code.

Recommended baseline architecture: a **TypeScript monorepo** with a responsive web client, cross-platform mobile clients, a modular backend/API layer, PostgreSQL, Redis, WebSockets, object storage, background workers, a payment provider, an identity/KYC provider, and Stockfish-based analysis/detection services. The backend must remain authoritative for game state, clocks, ratings, league points and financial state.

2. Product Goals & Scope

Goal	Definition of Done
Competitive chess	Players can find opponents, play synchronized legal games, reconnect, finish games and receive authoritative results.
Ratings	Rated games update Elo exactly once; rating history and statistics are queryable and auditable.
Paid leagues	Configurable leagues support entry fees, participant limits, rules, live standings, prize pools and controlled payouts.
Trust & safety	Suspicious play, multi-account patterns, reports and payout risk are detected and routed through automated/manual review.
Social	Players can add friends, invite players, message, block and report.
Coaching	Completed games can be analyzed using Stockfish with move classifications and explanations.
Operations	Admins can manage users, leagues, tournaments, financial workflows, moderation and audit logs.
Production readiness	Monitoring, backups, CI/CD, security controls and mobile/web release processes are documented and repeatable.

3. Target Platforms & Product Surfaces

- **Responsive Web:** desktop, tablet and mobile browsers; primary stack recommended as Next.js + React + TypeScript.
- **iOS:** native-feeling mobile experience using React Native/Expo where practical, sharing domain/API models with web.
- **Android:** same cross-platform client architecture with platform-specific integrations where required.
- **Admin Console:** responsive but desktop-first secured application, preferably a dedicated admin route/application with role-based access.
- **Backend:** cloud-hosted API, real-time gateway, workers and scheduled jobs.

4. Recommended Technical Architecture

The platform should use a layered, service-oriented modular monolith initially rather than immediately splitting into many microservices. This reduces operational complexity while preserving clear module boundaries. Real-time chess and asynchronous workloads should be independently scalable components.

Recommended stack

Layer	Recommended technology	Purpose
Web frontend	Next.js, React, TypeScript	Responsive player experience, SEO/public pages, authenticated app.
Mobile	React Native + Expo + TypeScript	iOS/Android app with shared product/domain logic.
UI	Tailwind CSS / design system; React/React Native	Consistent responsive UI and reusable components.
Backend API	Node.js + NestJS or Fastify + TypeScript	Auth, users, games, leagues, tournaments, payments, admin APIs.
Realtime	WebSockets / Socket.IO or ws	Game events, clocks, invitations, standings and presence.
Chess rules	chess.js or equivalent server-validated engine	Legal move validation, FEN/PGN handling, game termination.
Database	PostgreSQL	Transactional source of truth for users, games, ratings, leagues and finance.
Cache / coordination	Redis	Sessions/cache, presence, matchmaking queues, distributed locks, rate limits.
Queue / jobs	BullMQ or equivalent Redis-backed queue	Analysis, notifications, anti-cheat, payouts, reports and scheduled jobs.
Chess engine	Stockfish	Game analysis and anti-cheat signal generation.
Object storage	S3-compatible storage	Avatars, exports, evidence and generated artifacts.
Payments	Provider selected after country/compensation review	Player fees, refunds and payout rails.
KYC/identity	Provider selected by target territories	Winner verification, age/identity checks and risk screening.
Notifications	FCM/APNs + email provider	Push, transactional email and in-app notifications.
Observability	OpenTelemetry + centralized logs + metrics	Errors, delays, errors and operational diagnostics.
CI/CD	GitHub Actions/GitLab CI	Testing, security scans, builds and deployments.
Hosting	AWS/GCP/Azure or managed equivalent	Production infrastructure, scaling, backups and networking.

4.1 High-Level Component Diagram

Client Layer

Web (Next.js) | iOS/Android (React Native)

↓ HTTPS / WSS

Edge Layer

CDN + WAF + TLS + Rate Limiting + Load Balancer

↓

Application Layer

API Gateway / Backend → Auth → Users → Games → Matchmaking → Ratings → Leagues → Tournaments → Payments → Payouts → Social → Notifications → Admin → Anti-Cheat

↕

Realtime Gateway → Game rooms / clocks / presence / standings

↕

Data & Infrastructure

PostgreSQL + Redis + Job Queue + Object Storage

↓

External Services

Payment Provider + KYC/Identity + Email + APNs/FCM + Monitoring

4.2 Architectural Principles

- Backend-authoritative game state: clients never decide final results, ratings or prize eligibility.
- Transactional financial state: every money movement has an immutable ledger entry and idempotency key.
- Configuration over hard-coding: league rules and prize formulas are versioned configuration.
- Event-driven side effects: notifications, analytics, anti-cheat analysis and exports run asynchronously.
- Auditability: critical admin, financial, rating and moderation actions are logged.
- Least privilege: separate player, moderator, finance, support and super-admin permissions.
- Idempotency: retries must not duplicate games, ratings, charges, refunds or payouts.
- Graceful degradation: if analysis or notifications are delayed, gameplay should continue.

5. Database & Data Model

PostgreSQL is the system of record. Use UUIDs for primary identifiers, UTC timestamps, database constraints, explicit status fields and append-only records for financial/audit history. Avoid storing mutable financial balances as the only source of truth.

5.1 Core Entities

Entity	Key fields / relationships
users	id, email, password/auth provider reference, status, created_at, last_seen_at.
profiles	user_id, username, avatar_url, bio, country/territory where permitted, display preferences.
player_ratings	user_id, rating, provisional flag, games_count, deviation/metadata if Glicko is later adopted.
rating_history	game_id, user_id, old_rating, new_rating, delta, formula_version, created_at.
games	id, white_player_id, black_player_id, rated, time_control, status, result, termination, started_at, ended_at, PGN/FEN metadata.
game_moves	game_id, ply, player_id, SAN, UCI, FEN_after, server_timestamp, clock_remaining_ms.
game_events	game_id, sequence_no, event_type, payload, server_timestamp; useful for replay/audit.
matchmaking_tickets	user_id, rating, time_control, region, queue status, created_at.
leagues	id, name, status, starts_at, ends_at, participant_limit, entry_fee, currency, rule_config, prize_config.
league_memberships	league_id, user_id, paid status, joined_at, eligibility state, final_rank.
league_scores	league_id, user_id, score, eligible_games, Elo component, performance component, rule_version.
league_standings_snapshots	league_id, generated_at, ranked rows for historical/audit purposes.
tournaments	format, registration window, start/end, time control, capacity, status, rules.
tournament_entries	tournament_id, user_id, status, seed/rating snapshot.
tournament_rounds	tournament_id, round_no, pairing method, status.
tournament_matches	round_id, game_id, player pair, result, advancement.
friendships	requester_id, recipient_id, status, timestamps.
conversations / messages	conversation_id, participant IDs, message body, moderation status, timestamps.
blocks / reports	reporter, target, reason, evidence, status, moderator resolution.
payments	provider_payment_id, user_id, purpose, amount_minor, currency, status, idempotency_key.
ledger_entries	immutable debit/credit entries referencing payments/refunds/payouts.
payouts	winner, league/tournament, amount, KYC state, approval state, provider state.
refunds	payment reference, amount, reason, initiated_by, provider status.
identity_checks	user, provider reference, status, reviewed_at; do not store unnecessary sensitive documents.
notifications	user, type, payload, read_at, delivery state.
audit_logs	actor, action, resource, before/after metadata, IP/device metadata where legally appropriate, timestamp.
anti_cheat_cases	game/player, risk score, signals, status, reviewer, resolution.
league_rule_versions	league_id, version, JSON configuration, effective interval.

5.2 Important Database Rules

- Unique constraint on username; normalize case for lookup.
- Unique constraint on active league membership per user/league.
- Unique provider IDs for payment and payout records.
- Unique (game_id, ply) for moves and unique (game_id, event sequence) for event ordering.
- Use database transactions when completing a game: lock/finalize game → write result → update ratings → update league score inputs → emit domain event.
- Use row-level locking or equivalent transaction strategy to prevent double-finalization.
- Never delete financial/audit records; use status transitions and reversals.

6. Functional Requirements

6.1 User Accounts & Profiles

- FR-ACC-01: Register with email and secure authentication provider.
- FR-ACC-02: Verify email where required by policy.
- FR-ACC-03: Support login, logout, session expiry and device/session management.
- FR-ACC-04: Password recovery must use expiring, single-use tokens or managed auth provider flows.
- FR-ACC-05: Username must be unique and protected against impersonation/abuse.
- FR-ACC-06: Profile supports username, avatar, optional bio, rating, statistics and game history.
- FR-ACC-07: User can configure privacy, notification and account settings.
- FR-ACC-08: User status includes online, offline, in-game and optionally away; status must not expose unnecessary presence data.
- FR-ACC-09: Suspended/banned users cannot initiate restricted actions.
- FR-ACC-10: Account deletion/export workflows must comply with applicable privacy requirements while preserving legally required financial/audit records.

6.2 Chess Gameplay

- FR-GAME-01: Create a game with two players, time control, rated/unrated flag and unique game ID.
- FR-GAME-02: Server validates every move against authoritative game state.
- FR-GAME-03: Client renders legal moves and board state but cannot bypass server validation.
- FR-GAME-04: Clock uses server timestamps/monotonic timing strategy; client clock is presentation only.
- FR-GAME-05: Supported initial time controls should be configurable; recommended launch set: Bullet, Blitz, Rapid and custom presets.
- FR-GAME-06: Support resignation, draw offer/accept/decline, automatic draw conditions, checkmate, stalemate and timeout.
- FR-GAME-07: Reconnect must restore latest authoritative state and sequence number.
- FR-GAME-08: Duplicate/out-of-order WebSocket events must be safely ignored or reconciled.
- FR-GAME-09: Game completion writes PGN and immutable result metadata.
- FR-GAME-10: Spectating can be added later; if included in v1, spectator permissions must be read-only.

6.3 Real-Time Game Protocol

Example message model:

Client → **Server**: join_game, make_move, offer_draw, respond_draw, resign, ping, reconnect

Server → **Client**: game_snapshot, move_accepted, move_rejected, clock_update, draw_offer, game_finished, opponent_reconnected, error

Every event should include: **game_id**, **sequence_no**, **event_id**, **server_timestamp**, **payload**. The server rejects stale sequence numbers and the client requests a fresh snapshot when a gap is detected.

6.4 Matchmaking

- Primary ranking dimension: Elo rating.
- Secondary dimensions: time control, queue age, region/latency and optional rating tolerance.

- Rating tolerance expands gradually with queue wait time.
- Prevent a player from receiving multiple simultaneous active match assignments.
- Use Redis-backed queueing/locks; persist game creation in PostgreSQL.
- A match assignment must have an expiry/acceptance window to prevent abandoned reservations.

6.5 Elo Rating

Default recommended formula for a standard Elo implementation: $R' = R + K \times (S - E)$, where $E = 1 / (1 + 10^{((R_{\text{opponent}} - R)/400)})$. K should be configurable by rating band, provisional state or game count. Store the formula/version used for every rated result so historical calculations remain reproducible.

Rating manipulation controls should include provisional-rating limits, abnormal win-rate/match-pattern detection, opponent-collusion signals, repeated rematches, IP/device/account relationship signals where lawful, and review workflows. Detection should flag rather than automatically punish unless confidence is extremely high and policy permits it.

7. Paid Competitive Leagues

Leagues are the primary commercial/competitive differentiator. The implementation must separate **league configuration**, **eligibility**, **scoring**, **standings** and **financial settlement**. This allows rules to evolve without changing application logic.

7.1 League Lifecycle

- 1 Admin creates a draft league and defines name, dates, participant limit, entry fee, currency, eligible territories/ages, scoring configuration and prize configuration.
- 2 System opens registration only when configuration passes validation.
- 3 Player completes payment and required eligibility checks.
- 4 Membership becomes active after successful payment/verification.
- 5 League starts at configured time; only eligible members can earn competition points.
- 6 Each eligible game produces scoring inputs; standings update transactionally/asynchronously.
- 7 At league end, system freezes eligible results and computes final standings using a versioned rule set.
- 8 Top-ranked players enter payout review. Identity/risk checks must pass before payout approval.
- 9 Admin approves/rejects/holds payouts; provider executes payment.
- 10 League becomes archived with final standings, financial settlement and audit records.

7.2 Configurable Rule Model

Store a JSON configuration similar to:

```
rule_version: 1
elo_weight: 0.40
performance_weight: 0.35
activity_weight: 0.25
min_eligible_games: 10
max_eligible_games: 50
participant_limit: 30
prize_pool_percent: 80
platform_percent: 20
tie_breakers: [score, performance, rating_gain, games_played]
```

The actual production formula must be finalized with the product/legal/finance owner. Never silently change rules for an active league.

7.3 Prize Pool Accounting

For an illustrative league with N paid participants and entry fee F , gross collected amount is $N \times F$. If the configured prize pool percentage is P , prize pool = gross $\times P$. Platform share = gross – prize pool, subject to payment fees, taxes, refunds and applicable legal/accounting treatment. The specification does **not** assume that an 80/20 split is legally or commercially valid in every territory; it must remain configurable and be reviewed before launch.

7.4 Standings

- Show rank, player, score, eligible games, rating change and relevant scoring components.
- Standings should identify provisional/unverified or ineligible status without exposing private risk details.
- Use deterministic tie-break rules stored in the league configuration.
- Publish periodic snapshots for audit and historical display.
- Final standings become immutable after settlement except through a documented admin correction process.

7.5 Refunds & Disputes

- Refund eligibility depends on league status, payment status and published terms.
- Every refund requires a reason code and actor/system reference.
- Disputes create a case with evidence, status, SLA and resolution.
- A league can be paused or suspended by authorized admins.
- If a game is voided after league scoring, downstream points and payout eligibility must be recalculated through a controlled adjustment process.

8. Payments, Payouts, Identity & Compliance

Because ChessCoin combines paid competitive entry and prizes, payment architecture must be designed together with legal/compliance requirements. The engineering team must not launch paid leagues in a territory until the product owner and counsel confirm that the model is permitted and the provider supports it.

8.1 Provider Selection Criteria

Criterion	Required evaluation
Geography	Supported countries/states and payout destinations.
Business model	Whether skill-based competition with entry fees/prizes is supported.
Payouts	Winner payouts, onboarding, bank/card support, minimums and settlement timing.
KYC/AML	Identity verification, sanctions/risk checks and data retention requirements.
Age/territory	Ability to enforce age and location restrictions where required.
Fees	Payment processing, FX, payout and dispute fees.
Webhooks	Signed webhook support, retry behavior and event reconciliation.
Operational controls	Refunds, holds, disputes, chargebacks and reporting.

Implementation recommendation: shortlist a mainstream payment platform that supports the target territories and a marketplace/payout product where needed, then validate the exact ChessCoin business model with the provider before development is locked. Keep payment interfaces behind an internal adapter so the provider can be replaced without rewriting league logic.

8.2 Payment State Machine

created → pending → authorized/paid → failed/cancelled/refunded/partially_refunded. Provider webhooks are authoritative for final provider state, but every webhook must be signature-verified and idempotently processed.

8.3 Payout State Machine

eligible → KYC_pending → risk_review → admin_approved → provider_pending → paid OR rejected/held/failed. A payout must never be released solely because a client displays a winner.

8.4 Money Representation

- Store monetary values in integer minor units (e.g., cents/paise) plus ISO currency.
- Do not use floating-point arithmetic for financial amounts.
- Maintain immutable ledger entries and reconcile provider settlements.
- Use idempotency keys on charge/refund/payout requests.
- Record provider event IDs and raw payload references securely for reconciliation.
- Restrict financial administration to dedicated roles and log all actions.

9. Coach Mode

Use Stockfish as the initial engine. Analysis should run asynchronously so a slow engine does not block gameplay.

- Game submitted → analysis job queued → engine evaluates positions → move classification → summary generation → result stored.
- Classify moves into categories such as best/good/inaccuracy/mistake/blunder using configurable centipawn-loss thresholds.
- Show evaluation graph, key moments, suggested moves and variations.
- Provide educational explanations generated from structured engine signals; avoid claiming an engine score is a human explanation.
- Allow analysis depth/time limits to control infrastructure cost.
- Cache completed analysis by game/version/depth where practical.

10. Friends, Messaging & Moderation

- Friend request lifecycle: pending → accepted/rejected/cancelled.
- Direct invitation launches a game setup or challenge flow.
- Messaging supports pagination, unread counts, timestamps and basic moderation.
- Block prevents direct messaging/invitations and can hide the blocker from relevant discovery surfaces.
- Report flow supports user, game and message reports with reason categories and evidence.
- Rate-limit friend requests and messages to reduce spam.

11. Traditional Tournaments

Recommended v1 formats: **Swiss** and **single-elimination**. Swiss is suitable for larger competitive groups; single elimination is easier to operate and understand. Round-robin can be added when scheduling requirements are clear.

- Admin configures format, capacity, registration window, time control, start time, rating restrictions and tie-break rules.
- Registration validates eligibility and capacity.
- Pairing service generates deterministic pairings and stores them before publishing.
- Results come from authoritative game completion.
- Standings and advancement update after verified results.
- Admins can correct a result through an audited override workflow.

12. Notifications

- In-app notification center with read/unread state.
- Push notifications via APNs/FCM for supported mobile apps.
- Email for account security, league receipts, important league/tournament updates and payout status.
- Template-based notification service with localization-ready content.
- User notification preferences control non-essential notifications.

- Queue notifications asynchronously and use provider message IDs for troubleshooting.

13. Administrator Panel

Module	Capabilities
Dashboard	Users, active games, revenue/payment status, league metrics, suspicious activity, system health.
Users	Search, profile view, sessions, suspension/ban, account notes, reports, verification status.
Games	Search games, replay PGN, inspect moves, flags, anti-cheat cases, void/restore workflows.
Leagues	Create/edit draft, configure rules, monitor participants, pause/end, standings, settlement.
Tournaments	Create format, registration, pairings, schedules, result corrections, archive.
Finance	Payments, refunds, payout queue, provider statuses, reconciliation and exports.
Verification	KYC status, manual review state, winner verification outcomes; minimal sensitive data exposure.
Moderation	Reports, blocks, disputes, sanctions, evidence and resolution history.
Notifications	Templates, campaigns where permitted, delivery status and failed notifications.
Analytics	DAU/MAU, games, retention, match completion, league conversion, payment metrics.
Audit	Search immutable admin/security/financial actions.

13.1 RBAC Roles

Role	Example permissions
Super Admin	All configuration and access; emergency controls.
Operations Admin	Users, leagues, tournaments, notifications.
Moderator	Reports, chat moderation, suspensions within assigned scope.
Finance Admin	Payments, refunds, payout approvals, reconciliation.
Analyst	Read-only operational and financial analytics.
Support	Limited user/game lookup; no financial approval or ban authority.

Require MFA for all privileged accounts, short admin sessions, re-authentication for sensitive financial actions and complete audit logging.

14. Security Requirements

- TLS for all network traffic; secure cookies/session tokens where applicable.
- Password hashing must use a modern password-hashing algorithm through a trusted authentication provider/library; never store plaintext passwords.
- Server-side authorization on every protected endpoint.
- Input validation using DTO/schema validation and output encoding.
- Parameterized database access/ORM safeguards.
- CSRF protection for cookie-based browser flows.
- Content Security Policy, secure headers and clickjacking protection.
- Rate limits on login, password reset, messaging, friend requests, game actions and admin APIs.
- Secrets stored in a secrets manager, never in source control.
- Dependency and container vulnerability scanning in CI.
- Regular backups with restore testing.
- PII minimization, encryption at rest where supported, retention policies and access controls.
- Audit logs protected from ordinary application-user modification.

15. Anti-Cheating & Platform Protection

No anti-cheat system can guarantee perfect detection. ChessCoin should use a layered risk engine that combines game-level engine analysis, behavioral signals, account relationships and manual review.

15.1 Automated Signals

- Engine move agreement and centipawn-loss patterns over sufficient sample sizes.
- Time-to-move anomalies and suspiciously consistent timing.
- Opening/variation behavior and sudden rating-performance changes.
- Repeated opponents, coordinated results and suspicious match sequences.
- Multiple accounts sharing device/network signals where lawful and proportionate.
- Payment/KYC relationship anomalies for prize eligibility.
- High-confidence bot/automation indicators.
- Impossible or inconsistent client/game protocol behavior.

15.2 Review Workflow

Signals → risk score → case creation → evidence collection → moderator review → warning/restriction/suspension/ban or cleared. Financial prize payouts can be held while a case is open. Automated actions should be conservative and policy-driven.

16. Reliability & Resilience

- Target game-state availability: 99.9% monthly after production stabilization.
- API health checks and readiness/liveness endpoints.

- WebSocket reconnect with exponential backoff and snapshot recovery.
- Database automated backups and periodic restore drills.
- Redis failure must not corrupt authoritative game/financial state.
- Background jobs use retries with dead-letter/failed-job inspection.
- Graceful handling of payment-provider downtime; do not mark payments successful based on client callback alone.
- Disaster recovery targets (RPO/RTO) must be agreed before production launch; recommended initial target RPO ≤ 15 minutes and RTO ≤ 2 hours for core services.

17. API Specification

Use REST for standard CRUD/transaction APIs and WebSockets for real-time gameplay. Version public APIs (e.g., /api/v1). Generate OpenAPI documentation from server schemas.

Area	Representative endpoints
Auth	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/forgot-password
Profile	GET /users/me, PATCH /users/me, GET /players/:username, GET /players/:id/stats
Games	POST /games, GET /games/:id, POST /games/:id/moves, POST /games/:id/resign, POST /games/:id/draw-offer
Matchmaking	POST /matchmaking/tickets, DELETE /matchmaking/tickets/:id
Ratings	GET /players/:id/rating-history, GET /leaderboards
Leagues	GET /leagues, GET /leagues/:id, POST /leagues/:id/join, GET /leagues/:id/standings
Payments	POST /payments/checkout, GET /payments/:id, POST /payments/webhooks/provider
Payouts	GET /payouts/me, admin payout endpoints
Coach	POST /games/:id/analysis, GET /games/:id/analysis
Social	POST /friends/requests, GET /friends, POST /conversations, POST /messages
Tournaments	GET /tournaments, POST /tournaments/:id/register, GET /tournaments/:id/standings
Admin	Admin-scoped routes for users, games, leagues, tournaments, finance, reports, analytics and audit.

API conventions: consistent error codes, request IDs, pagination, sorting/filtering, validation schemas, rate-limit headers, authorization checks and idempotency keys for state-changing financial operations.

18. WebSocket Game API Contract

Connection: authenticated WSS connection.

Room: game:{gameId}

Required handshake: authenticated user + game authorization.

Move payload: gameId, clientEventId, expectedSequence, move (UCI/SAN), clientTimestamp.

Server response: sequence, accepted/rejected, authoritative FEN, clocks, result/status.

Recovery: client sends reconnect with lastSequence; server sends missed events or full snapshot.

19. UI/UX Requirements

The interface should feel like a modern competitive chess product: fast, focused, accessible and visually consistent across responsive web and mobile.

Screen / Flow	Requirements
Landing	Clear value proposition, play CTA, leagues, tournaments, coaching and trust/legal links.
Dashboard	Quick Play, current games, rating, recent games, league progress, tournaments, friends.
Chessboard	Large board, legal-move affordances, clocks, captured pieces, move list, draw/resign controls, reconnect state.
Profile	Avatar, username, rating, stats, history, achievements if included.
League	Entry fee, rules, dates, seats, live standings, score breakdown, prize information, progress.
Tournament	Format, schedule, registration, bracket/round pairing, standings and results.
Coach	Game replay, evaluation graph, mistakes, suggested lines and learning summary.
Messages	Conversation list, unread state, report/block actions.
Payments	Review order, provider checkout, receipt, status and refund terms.
Admin	Dense data tables, filters, bulk actions with confirmation, role-aware controls.

Accessibility: keyboard navigation on web, sufficient contrast, focus states, screen-reader labels for controls, responsive touch targets and reduced-motion support. Chessboard coordinates and move information should remain understandable without relying solely on color.

20. Non-Functional Requirements

Category	Requirement
Performance	Typical API p95 target < 300 ms excluding external providers; game move acknowledgement target < 150 ms under normal conditions.
Scalability	Stateless API nodes; horizontally scalable WebSocket gateway; Redis coordination; PostgreSQL read replicas when needed.
Availability	Core target 99.9% monthly after stabilization.
Security	OWASP-aligned secure development, dependency scanning, secrets management, MFA for admins.
Observability	Metrics, structured logs, traces, error tracking, alerts and business KPIs.
Maintainability	TypeScript, linting, formatting, unit/integration/e2e tests, code review and modular architecture.
Localization	Text and currency architecture ready for multiple locales; legal content configurable.
Privacy	Data minimization, access controls, deletion/export workflows and documented retention.
Recovery	Automated backups, restore tests and documented incident/runbook procedures.

21. Testing & Quality Assurance

21.1 Test Pyramid

- Unit tests: rating formulas, scoring rules, chess-game services, state machines, permissions.
- Integration tests: PostgreSQL transactions, Redis queues, payment webhooks, KYC callbacks, notifications.
- End-to-end tests: registration → matchmaking → game → result → rating → league score → payout workflow.
- Realtime tests: reconnect, duplicate events, out-of-order messages, simultaneous actions and timeout boundaries.
- Security tests: authentication/authorization, injection, XSS, CSRF, rate limiting, insecure direct object references and dependency vulnerabilities.
- Load tests: concurrent games, matchmaking bursts, standings updates and WebSocket connections.
- Mobile tests: iOS/Android device matrix, background/foreground transitions, poor network, push notifications.
- Browser tests: current Chrome/Safari/Firefox/Edge plus responsive breakpoints.

21.2 Critical Acceptance Scenarios

Scenario	Expected outcome
Player makes legal move	Both players receive same authoritative position and clocks; move sequence increments once.
Player sends illegal move	Server rejects it; opponent state is unchanged.
Connection drops	Game continues according to server clock; reconnect restores exact state.
Timeout	Server determines timeout according to clock state and finalizes result once.
Rated game ends	Result stored once; Elo changes once; history updated.
League entry payment succeeds	Membership activated only after verified provider state.
Payment webhook retried	No duplicate membership/ledger effect.
League ends	Standings freeze using versioned rules; winners determined deterministically.
Suspicious winner	Payout is held; KYC/risk review case created.
Admin changes score rule	New rule version applies only to allowed league state; active league rule version remains immutable.
Refund	Provider refund and internal ledger reconcile; league eligibility updates according to policy.

22. Production Infrastructure & DevOps

22.1 Environment Strategy

- **Local:** Docker Compose for PostgreSQL, Redis and optional Stockfish; seeded test data.
- **Development:** shared cloud dependencies with isolated database/storage and test payment mode.
- **Staging:** production-like infrastructure, test payment/KYC providers, release candidate validation.
- **Production:** isolated account/project, private database/networking, managed secrets, backups and monitoring.

22.2 CI/CD Pipeline

- 1 Install dependencies with lockfile enforcement.
- 2 Lint, type-check and unit tests.
- 3 Build web/mobile/backend artifacts.
- 4 Run dependency and secret scanning.
- 5 Run integration tests against ephemeral services.
- 6 Build and scan containers if containerized.
- 7 Deploy to staging on approved branch/tag.
- 8 Run smoke/e2e tests.
- 9 Require approval for production.
- 10 Deploy with rollback strategy and database migration safeguards.

22.3 Monitoring

- API latency/error rate, WebSocket connections/disconnections, active games and move rejection rate.
- Matchmaking queue size and match creation latency.
- Payment success/failure/webhook lag and payout failures.
- League standings job duration and failed jobs.
- Database CPU/connections/slow queries/storage.
- Redis memory/evictions/latency.
- Mobile crash-free sessions and push delivery failures.
- Security alerts and suspicious activity volume.

23. Development Phases & Deliverables

Phase	Scope	Primary deliverables
1	Discovery & architecture	Current-project assessment, final stack, domain model, UX audit, ADRs, backlog.
2	Foundation	Repo/monorepo, CI/CD, environments, auth, users, profiles, RBAC, DB migrations.
3	Chess core	Chessboard, rules, real-time game service, clocks, reconnection, matchmaking, result recording.
4	Ratings	Elo engine, history, leaderboards, anti-manipulation foundations.
5	Leagues	Registration, payment integration, rule engine, standings, scoring, prize calculation.
6	Finance & verification	KYC, payout workflow, refunds, disputes, ledger, reconciliation.
7	Coach & social	Stockfish analysis, friends, messages, blocks/reports, notifications.
8	Tournaments	Swiss/single elimination, pairings, scheduling, standings, history.
9	Security & QA	Anti-cheat, penetration/security testing, load testing, device/browser testing, bug fixing.
10	Launch	Production deployment, app-store builds/submission, monitoring, backups, runbooks, documentation.

23.1 Developer Work Breakdown

- Create architecture decision records (ADR) for backend framework, realtime protocol, auth, payment provider, KYC provider and hosting.
- Create database ERD and migrations before feature implementation.
- Define API contracts and shared TypeScript schemas.
- Build core game state machine before UI polish.
- Implement league rules as a pure, testable domain module.
- Implement finance as a ledger/state-machine module isolated from UI.
- Build anti-cheat signals as asynchronous jobs and an admin case-management workflow.
- Create test fixtures for games, ratings, leagues and payment states.
- Complete observability before production load testing.

24. Release Acceptance Criteria

- A new user can register, authenticate, create a profile and play a real-time game.
- Two players see identical authoritative board state and final result.
- Games recover correctly after temporary network loss.
- Rated games update Elo exactly once and display rating history.
- A configured paid league accepts eligible entry payments and maintains live standings.
- Prize pool and platform share are calculated from versioned configuration and reconciled with financial records.
- Winners can be placed into payout review and payouts can be approved/held/rejected with audit records.
- Refunds and disputes are supported end-to-end.
- Stockfish analysis works asynchronously for completed games.
- Friends, messaging, block and report workflows work with moderation controls.
- At least the agreed tournament formats can be created and completed.
- Push/in-app/email notifications work for defined critical events.
- Admin RBAC prevents unauthorized financial/security actions.
- Critical security and load tests pass.
- Production monitoring, alerts, backup and restore procedures are verified.
- Web production deployment is complete and mobile builds are prepared/submitted through the applicable stores.
- Source code, environment documentation, API docs, database schema, runbooks and deployment instructions are handed over.

25. Technical Documentation Handover

- Architecture diagram and ADRs.
- ERD/database schema and migration instructions.
- OpenAPI/API documentation and WebSocket protocol documentation.
- Environment variable reference without exposing secret values.
- Local development setup.
- CI/CD and deployment instructions.
- Payment/KYC integration and webhook documentation.
- League scoring/rule configuration documentation.
- Anti-cheat operations guide.
- Monitoring dashboards and incident runbooks.
- Backup/restore and disaster recovery procedure.
- Admin user guide and role matrix.
- Mobile release/signing/submission documentation.
- Known limitations and technical debt register.

26. Open Decisions Requiring Product/Legal Approval

Decision	Why it matters
Launch countries/territories	Determines payment, prize, age and competition-law constraints.
Minimum age	Must be enforced consistently with verification and legal policy.
League entry model	Subscription vs per-competition fee changes payment and eligibility logic.
Exact scoring formula	Must be frozen/versioned before each competition begins.
Prize distribution	Percentages and tie handling affect financial settlement.
Tournament v1 formats	Controls implementation scope.
KYC threshold	Whether all players or only winners need verification.
Anti-cheat policy	Defines automatic actions vs manual review and appeal process.
Retention/privacy	Defines data lifecycle and audit storage.
Payment/KYC provider	Must be validated against business model and target territories.

Appendix A — Suggested Repository Structure

```
apps/  
  web/ # Next.js responsive web  
  mobile/ # React Native / Expo iOS + Android  
  admin/ # Admin console (or protected web app)  
  api/ # Backend API + WebSocket gateway  
  worker/ # Background jobs / Stockfish / notifications  
  
packages/  
  domain/ # Game, rating, league, tournament domain rules  
  db/ # Schema, migrations, repositories  
  api-contracts/ # OpenAPI/Zod/shared DTOs  
  ui/ # Shared web UI components  
  mobile-ui/ # Shared mobile components  
  config/ # ESLint, TypeScript, environment schemas  
  
infra/  
  terraform/ # Infrastructure as code  
  docker/ # Local/production images  
  monitoring/ # Dashboards/alerts  
  
docs/  
  architecture/ # ADRs, diagrams  
  api/ # API and realtime docs  
  runbooks/ # Operations/security/recovery
```

Appendix B — Key State Machines

Domain	States
Game	created → waiting → active → completed → archived/voided
Matchmaking	queued → matched → accepted → expired/cancelled
League	draft → scheduled → registration_open → active → settling → completed → archived/suspended
Payment	created → pending → paid/failed/cancelled → refunded/partially_refunded
Payout	eligible → KYC_pending → review → approved → provider_pending → paid/failed/held
Tournament	draft → registration → locked → active → completed → archived
Moderation case	open → investigating → actioned/cleared → appealed → closed

Appendix C — Important Implementation Notes

- Do not let clients calculate authoritative clocks, results, Elo or prize amounts.
- Use a server-side monotonic time source for elapsed game time and persist enough information to reconstruct state.
- League scoring should be a deterministic pure function of frozen inputs and a versioned rule configuration.
- Every external provider callback must be verified, idempotent and auditable.
- Use feature flags for incomplete modules and staged rollout.
- Keep paid leagues disabled by territory until compliance approval is recorded.
- Treat anti-cheat as a risk system, not a single engine-comparison score.
- Design the system so Stockfish workers, WebSocket gateways and API nodes can scale independently.

End of Specification

This document is intended to serve as the baseline engineering specification. Final provider selection, legal/compliance policy, scoring formula, exact UI designs and launch territories should be approved during discovery before production implementation.